

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0510 Slot A

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 10%

QUESTIONS:4 Total Marks 20 Duration :60 Minutes Annexure A
Mock Interview

Code of Conduct:

I D.HUSSAINIAH(192425300) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D.HUSSAINIAH

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	

Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	
-----------------------	----	--	---	------------------------------------	---------------------------------	--

Question 1: A video stream drops frames during processing. Analyze the problem and redesign your approach to ensure smooth playback, explaining trade-offs and optimization techniques.

1. Root-Cause Analysis

Frame drops happen when the per-frame processing time exceeds the time budget dictated by the source frame rate (e.g., 33 ms per frame at 30 FPS). The usual causes are:

- Heavy per-frame computation (deep-learning inference, multiple filters, high-resolution processing).
- Single-threaded, synchronous pipeline where capture, processing, and display block each other.
- I/O bottlenecks — reading from disk/network slower than the display rate.
- Buffer mismatch — the capture buffer fills up while the consumer is still busy, so OpenCV's VideoCapture silently drops frames.

2. Redesigned Architecture

Move from a single blocking loop to a multi-threaded producer–consumer pipeline:

- Capture Thread: continuously reads frames using `cv2.VideoCapture` and pushes them into a thread-safe bounded queue (e.g., `queue.Queue(maxsize=N)`).
- Processing Thread(s): pop frames from the queue, run the CV/ML pipeline, and push results into an output queue. Use a thread/process pool for CPU-bound work.
- Display/Output Thread: consumes processed frames at a steady pace, independent of processing spikes.

```
cap_thread -> [input_queue] -> processing_pool -> [output_queue] -> render_thread
```

3. Optimization Techniques

- Frame skipping / adaptive sampling: process every Nth frame when the queue backlog grows, and interpolate or hold the last known result for skipped frames.
- Resolution scaling: downscale frames before heavy inference (`cv2.resize`) and upscale only the results (e.g., bounding boxes) back to original coordinates.

- Hardware acceleration: use GPU-backed OpenCV builds (CUDA), or offload inference to TensorRT/ONNX Runtime for real-time throughput.
- Bounded queues with drop policy: if the queue is full, drop the oldest frame instead of blocking capture — keeps playback synchronized with real time.
- Batching: batch multiple frames for GPU inference to better utilize parallelism.

4. Trade-offs

- Accuracy vs. Latency: skipping frames or downscaling reduces detection accuracy but keeps playback smooth.
- Memory vs. Smoothness: larger queues absorb bursts but increase latency and memory use.
- Complexity vs. Robustness: multi-threading adds synchronization complexity but is essential for real-time performance.

Conclusion: A producer–consumer threaded pipeline with adaptive frame skipping and GPU acceleration is the standard, interview-ready answer — it directly addresses the bottleneck while making the accuracy/latency trade-off explicit.

Question 2: You need to process only every 5th frame of a video. Design an efficient solution and justify your approach, considering performance and accuracy.

1. Approach

The naive method — reading every frame with `cap.read()` and discarding four out of five — is wasteful because VideoCapture still decodes every frame internally. Two efficient strategies exist:

Method A: Frame-index skipping with `CAP_PROP_POS_FRAMES`

```
cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
ret, frame = cap.read()
frame_idx += 5
```

- Directly seeks to the desired frame index instead of decoding intermediate frames.
- Efficient for formats/containers with reliable keyframe indexing (works best with I-frame-heavy codecs).
- Caveat: seeking on some compressed formats (long-GOP H.264) can still force nearby keyframe decoding, so it is not always faster than sequential skipping.

Method B: Sequential read with a modulo counter

```
count = 0
while True:
    ret, frame = cap.read()
    if not ret: break
    if count % 5 == 0:
        process(frame)
    count += 1
```

- Reads every frame sequentially (cheap decode) but only runs the expensive processing step on every 5th frame.
- More reliable across codecs since it avoids unpredictable seek behavior; the decode cost is far lower than the processing cost, so this is usually the efficient choice in practice.

2. Recommended Design

Use Method B (sequential decode + conditional processing) as the default, since decoding is cheap relative to CV/ML inference, and it guarantees correctness across all codecs. Use

CAP_PROP_POS_FRAMES seeking only for short-GOP formats or when random access to specific timestamps is required.

3. Performance vs. Accuracy Trade-off

- Performance gain: processing load drops to 20% of the original (1 in 5 frames), enabling real-time throughput on limited hardware.
 - Accuracy impact: fast-moving objects or short-duration events between sampled frames may be missed — acceptable for tasks like periodic monitoring, but risky for motion-critical tasks (e.g., tracking).
 - Mitigation: combine with interpolation or a lightweight motion detector on skipped frames to flag frames that need immediate (non-skipped) processing.
-

Question 3: A system must process both images and videos using the same pipeline. Propose a generalized design and explain how it handles both cases effectively.

1. Key Insight

A video is simply a sequence of images (frames). If the pipeline is designed around a common 'frame source' abstraction, the same processing logic can serve both inputs without duplication.

2. Generalized Architecture

- **Input Adapter Layer:** a unified `FrameSource` interface with two implementations — `ImageFrameSource` (yields a single frame) and `VideoFrameSource` (wraps `cv2.VideoCapture` and yields frames in a loop).
- **Core Processing Module:** a stateless function `process_frame(frame)` containing all CV logic (preprocessing, detection, analysis) — identical for both input types since it only ever sees one frame at a time.
- **Output Handler:** writes a single annotated image for image input, or accumulates frames into an output video (`cv2.VideoWriter`) for video input.

```
for frame in frame_source:    # image -> 1 iteration, video -> N iterations
    result = process_frame(frame)
    output_handler.write(result)
```

3. Implementation Notes

- Use Python generators (`yield`) so both sources expose the same iterable interface — the core pipeline is agnostic to how many frames arrive.
- Detect input type automatically from file extension or MIME type and instantiate the matching `FrameSource`.
- Keep processing stateless where possible (e.g., a detector) so it works identically per-frame; state that spans frames (like trackers) should be an optional pluggable component activated only for `VideoFrameSource`.

4. Benefits

- **Code reuse:** one processing module maintained instead of two separate codebases.
- **Extensibility:** adding a new input type (e.g., a live camera stream or RTSP feed) only requires a new `FrameSource` adapter, not changes to the core logic.

Question 4: You need to extract frames from a video and save only those with significant changes. Analyze the problem and design a method, explaining your decision criteria and approach.

1. Problem Analysis

Saving every frame wastes storage and downstream processing on near-duplicate, redundant content (e.g., static scenes). The goal is to keep only frames that represent a meaningful visual change — useful for surveillance summarization, keyframe extraction, or highlight generation.

2. Decision Criteria for 'Significant Change'

- Pixel-difference thresholding: compute the absolute difference between consecutive grayscale frames (`cv2.absdiff`) and compare the mean/sum against a threshold.
- Structural similarity (SSIM): more robust than raw pixel difference — tolerant to minor noise/lighting flicker while still catching real structural changes.
- Histogram comparison: compare color histograms (`cv2.compareHist`) between frames — robust to small object motion, sensitive to scene/lighting changes.
- Optical flow magnitude: for motion-focused use cases, large average flow vectors indicate significant movement.

3. Proposed Method

```
prev = None
for frame in video_frames:
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    gray = cv2.GaussianBlur(gray, (21,21), 0)
    if prev is not None:
        diff = cv2.absdiff(prev, gray)
        score = diff.mean()
        if score > THRESHOLD:
            save(frame)
    prev = gray
```

- Step 1 — Preprocess: convert to grayscale and apply Gaussian blur to suppress sensor noise and minor lighting flicker.
- Step 2 — Compare: compute frame-to-frame difference (`absdiff`, SSIM, or histogram correlation, depending on required sensitivity).

- Step 3 — Threshold decision: if the change score exceeds a calibrated threshold, mark the frame as significant and save it; otherwise discard it.
- Step 4 — Update reference: the saved/current frame becomes the new baseline for the next comparison, so gradual drift is still detected over time.

4. Design Considerations

- Threshold calibration: too low → floods output with near-duplicate frames; too high → misses real events. Tune empirically per use case, or use an adaptive/statistical threshold (mean + $k \cdot \text{std}$ of recent scores).
 - Noise robustness: blurring and using SSIM instead of raw pixel diff prevents false positives from compression artifacts or slight camera jitter.
 - Reference strategy: compare against the immediately previous frame for continuous change detection, or against a fixed 'last-saved' frame to detect cumulative drift instead of frame-to-frame micro-changes.
 - Efficiency: operate on downscaled grayscale frames for the comparison step, applying the decision only before running any expensive downstream processing.
-